Intrusion Tolerance Concepts

MSI – MEI – MI – MDS

2024/2025

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Outline

- Preliminary Observations
- Concepts
  - Fault models
  - Methodologies
  - Error Processing
- Frameworks
- Strategies

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

What measures the risk of intrusion?

- **RISK is a combined measure of the level of threat to which a system is exposed, and the degree of vulnerability it possesses:**

RISK* = *VULNERABILITY* ✘ *THREAT

- ***The correct measure of how potentially insecure a system can be (in other words, of how hard it will be to make it secure) depends:***
  - *on the number and severity of the flaws of the system (vulnerabilities)*
  - *on the potential of the attacks, it may be subjected to (threats)*

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Should we bring the risk to zero?

- Can we remove every flaw of the system (vulnerabilities) or avoiding all attacks it may be subjected to (threats)?
- Probably no:
  - it might be too costly and infeasible

    e.g., formal verification or disconnection from the internet
  - certain attacks come from the kind of service being deployed

    e.g., HTTP servers are always subject to attacks on port 80
  - certain vulnerabilities are attached to the design of the system

    e.g., the use of some technology or module for compatibility reasons
- If we work on an all-or-nothing perspective, every time we cannot ensure something is completely secure, we have a problem with representation
  - Can't we talk about acceptable risk?
  - Doesn't the hacker also incur in a cost of intruding?

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Assumption and Coverage

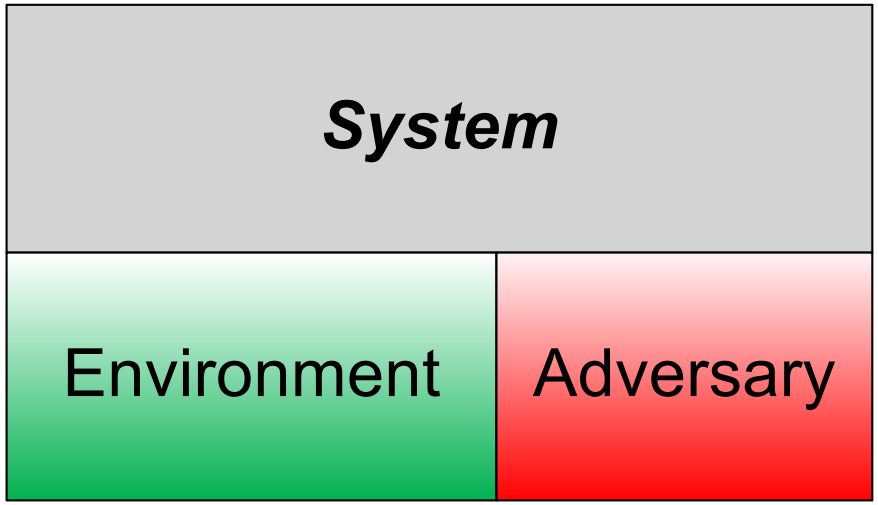
- **What are we missing?**
  - **we need to leave the all-or-nothing perspective**
  - **we need to represent *"more or less secure" formally***

- ***Separate concerns***
  - **between *specification* and *implementation***
- **Provide a notion of *coverage***
  - **Property A specifies a good thing about system S**
  - **System S is implemented to satisfy A**
  - **Implementation meets A with a confidence of Pr**
  - ***Coverage of A being provided by S is Pr***

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Assumption and Coverage

- **The notion of coverage is not only important for system components, but also to the environment the system will run (including the adversary we consider)**
  - **A well-designed *System* assume certain properties from the Environment and capabilities from the Adversary**
  - **These properties and capabilities only make sense if satisfied with high coverage by the system's target deployment scenario**



©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Trust and Trustworthiness
(support separation of concerns)

- **Trust:** the accepted dependence of a component, on a set of properties (functional or non-functional) of another component, subsystem or system
  - a trusted component has a set of properties that are relied upon by another component (or components)
  - if A trusts B, then A accepts that a violation in those properties of B might compromise the correct operation of A

- **Trustworthiness:** the measure in which a component, subsystem or system meets a set of properties (functional or non-functional)
  - trustworthiness of B measures the *coverage* of the trust of A

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Trusted vs. Trustworthy

Thou shall not trust non-trustworthy components!

- **B is trustworthy in the measure its properties are met**
  - **... and that coverage is < 1 in real systems...**
- **B should be trusted only to the extent of its trustworthiness**
  - **trust may have several degrees, quantitatively or qualitatively**
  - **related not only with security-related properties (e.g., timeliness)**
  - **trust and trustworthiness lead to complementary aspects of the <u>specification/design</u> and <u>implementation/verification</u> process**
- **we should talk about trusted-trustworthy components**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Outline

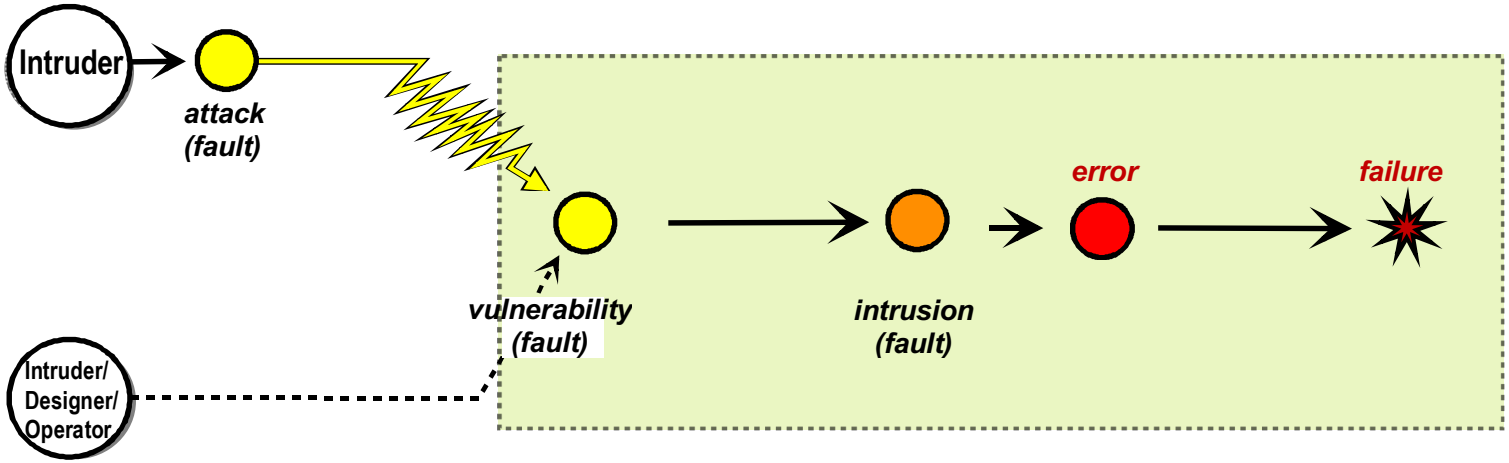
- Preliminary Observations
- Concepts
  - Fault models
  - Methodologies
  - Error Processing
- Framework
- Strategies

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Attack-Vulnerability-Intrusion composite fault model

Recall: attack + vulnerability → intrusion → error → failure

A specialization of the generic *"fault, error, failure"* sequence



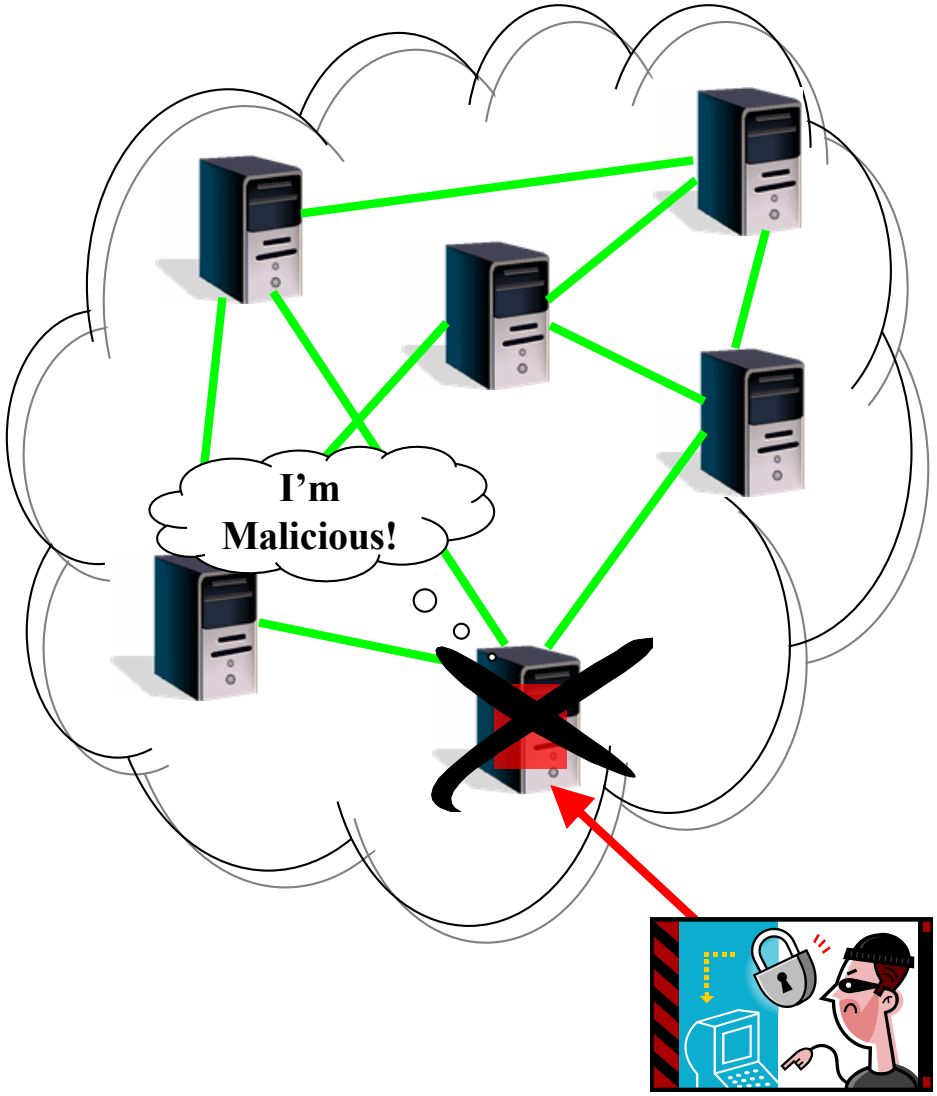
AVI sequence

Ciências ULisboa
©2010-25 P. Veríssimo and A. Bessani – All rights reserved.
INTOL

Cascading Faults through error propagation

(represent e.g., an intrusion campaign)

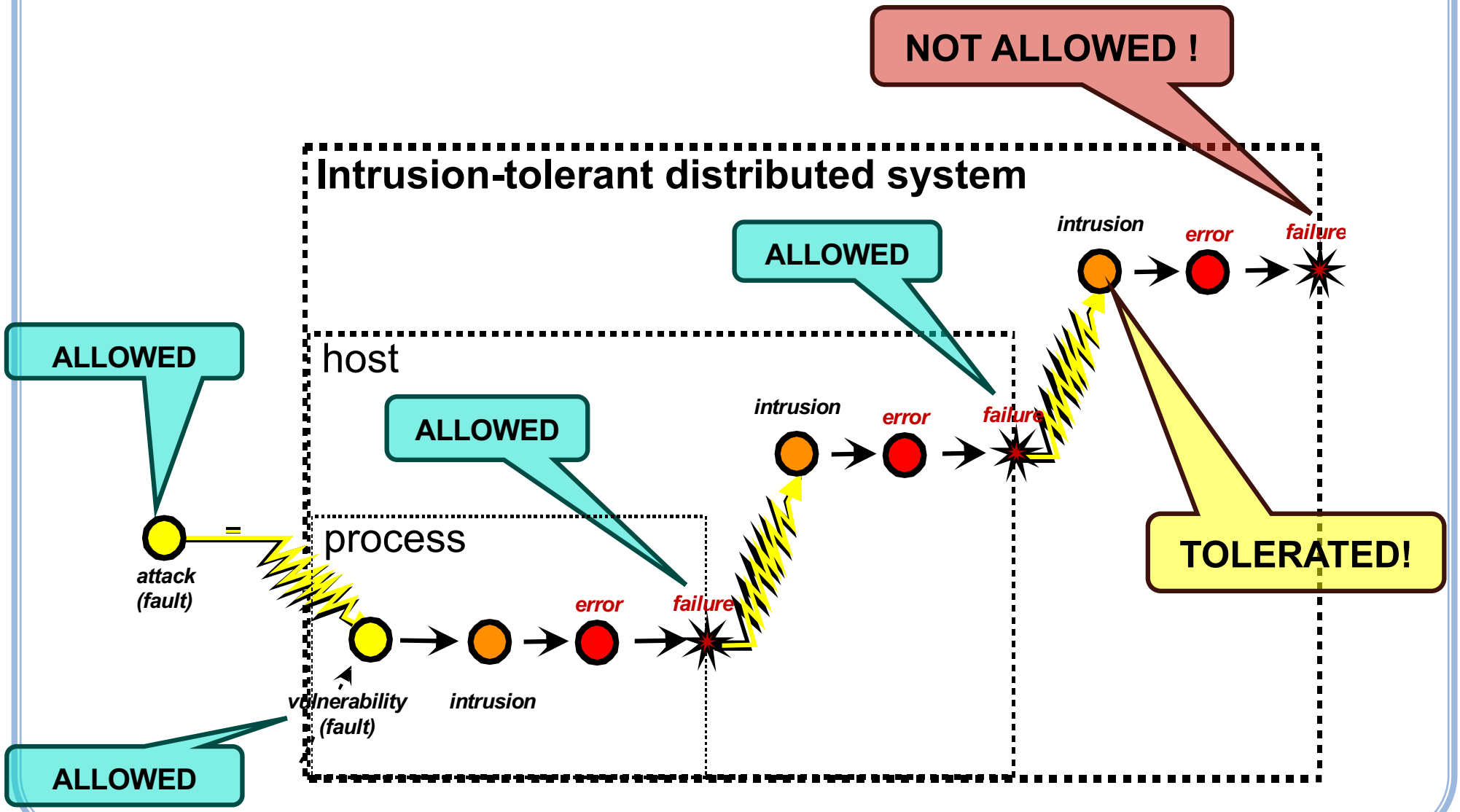
- Faults caused by an intrusion can be propagated through the whole system
- Consider the following intrusion campaign:
  1. Malicious hacker launches an attack against some host on a distributed system
  2. He compromises some service (a process) on this host
  3. With the permissions of this process it takes control of the machine
  4. At this point he tries to take control of the distributed system



©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Cascading Faults through error propagation

(an AVI interpretation)



Ciências ULisboa

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

INTOL

As for Fault Models…

- Although it is desirable to specify our fault model as detailed as possible, it is not practical
- Moreover, <u>how can we specify the malicious human behavior of an "intelligent" attacker</u>?
- So, for most intrusion-tolerant systems, we assume that machines can be compromised
  - In principle, the cause of the intrusion does not matter
  - It is also difficult to predict what an intruded machine will do
- Intruder-controlled machines can be seen as components subject to Arbitrary (Byzantine) faults
  - A compromised machine can do anything [but break cryptography or delay/drop messages forever (typical adversary limitation)]
  - It allows maximum coverage, because nothing is assumed

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Outline

- Preliminary Observations
- Concepts
  - Fault models
  - Methodologies
  - Error Processing
- Framework
- Strategies

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

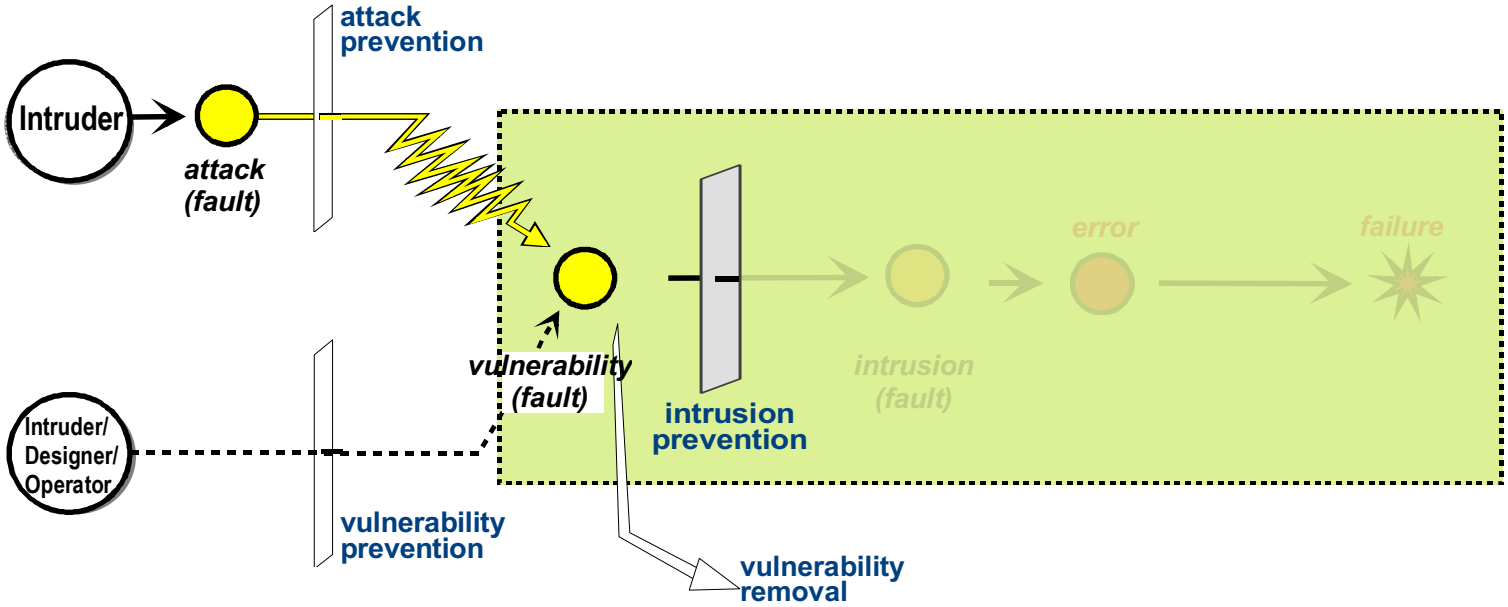
Achieving trustworthiness w.r.t. malicious faults
(the classical ways...)

- **Attack prevention**
  - Ensuring attacks do not take place against certain components
- **Attack removal**
  - Taking measures to discontinue attacks that took place
- **Vulnerability prevention**
  - Ensuring vulnerabilities do not appear in certain components
- **Vulnerability removal**
  - Eliminating vulnerabilities in certain components (e.g., bugs)

INTRUSION PREVENTION

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Avoiding security failure
canonical track: intrusion prevention

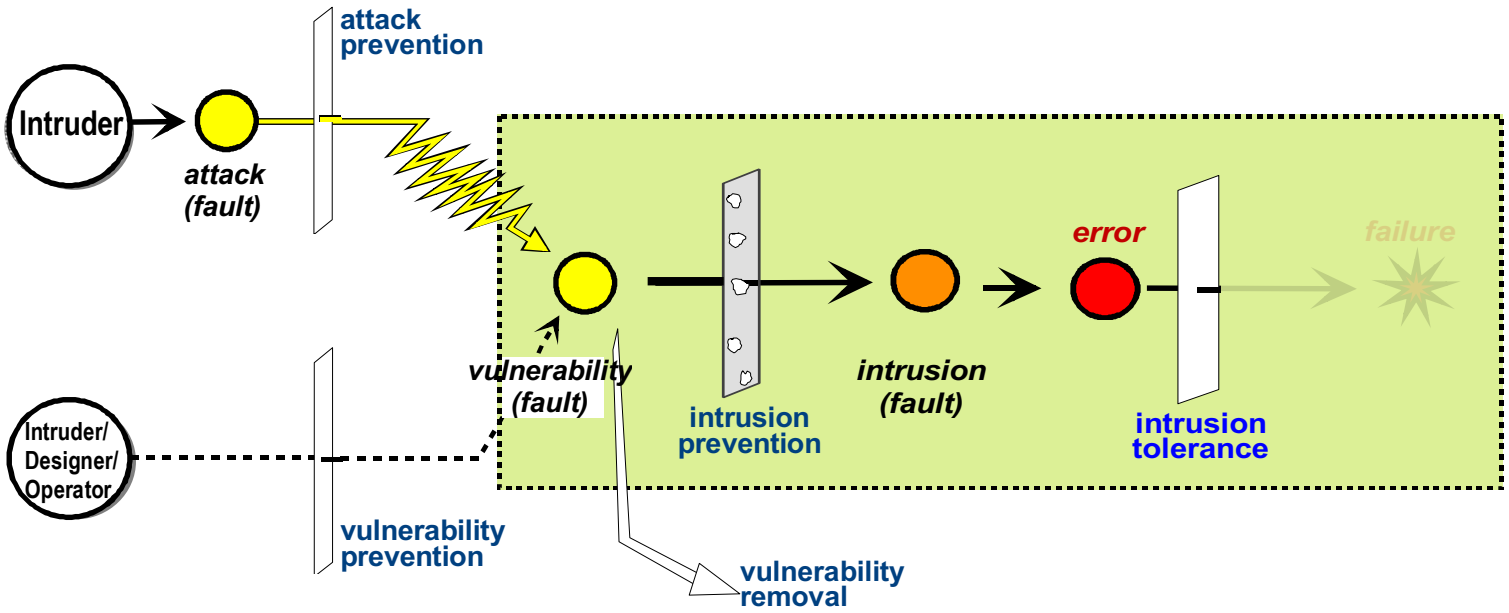


- All of them are imperfect…

Ciências ULisboa
©2010-25 P. Veríssimo and A. Bessani – All rights reserved.
INTOL

Avoiding security failure

less canonical track: intrusion tolerance



Ciências ULisboa

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

When and where to use intrusion tolerance?

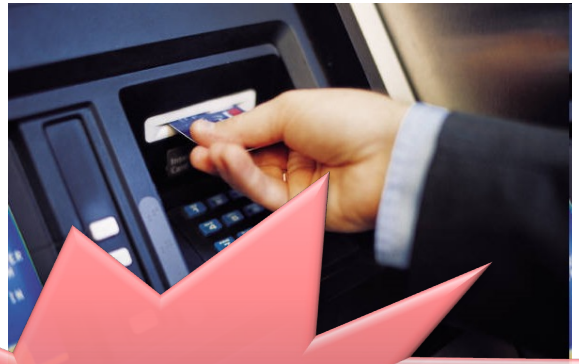
- *Intrusion tolerance* is a methodology for achieving trustworthiness systematically
- The objective is to complement classical techniques related with intrusion prevention
- A trusted-trustworthy component *A* which is intrusion tolerant tends to have its properties satisfied with a higher coverage
- It means that other components and systems can trust A with more confidence
- Its also useful for implementing decentralized application spanning multiple administrative domains (distributed trust)
- Intrusion tolerance is a relatively expensive technique, typically reserved for critical systems

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

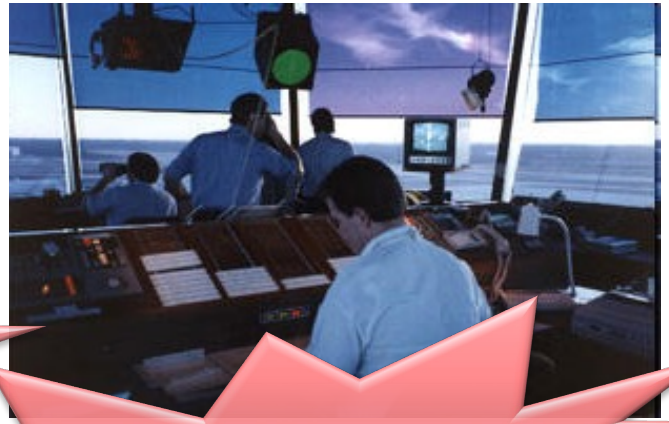
When and where to use intrusion tolerance?

https://www.hyperledger.org

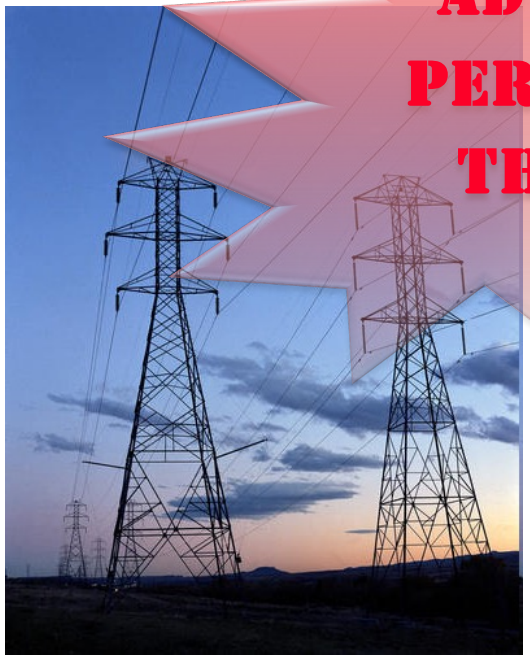
financial systems



https://bitcoin.org



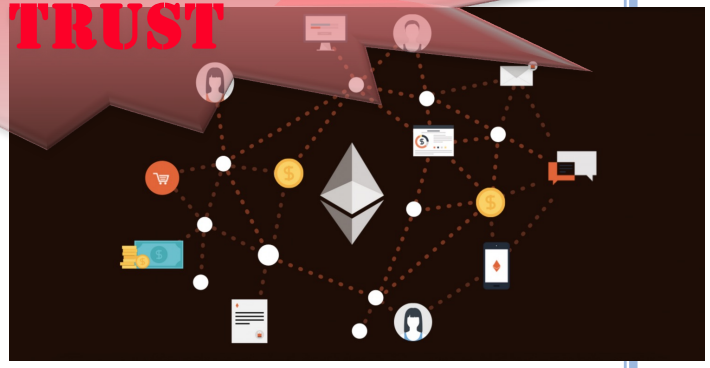
transmission grid



ADVANCED PERSISTENT THREATS

DISTRIBUTED TRUST

power generation



decentralized applications

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

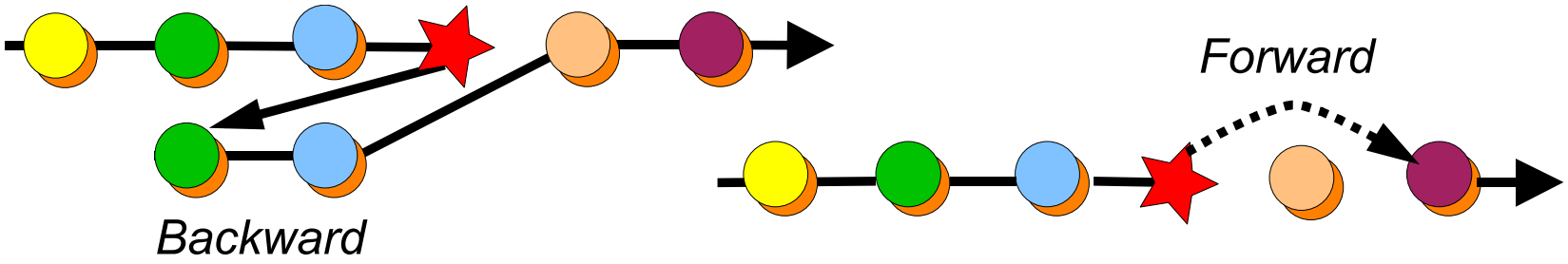
Outline

- Preliminary Observations
- Concepts
  - Fault models
  - Methodologies
  - Error Processing
- Framework
- Strategies

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Processing the errors deriving from intrusions

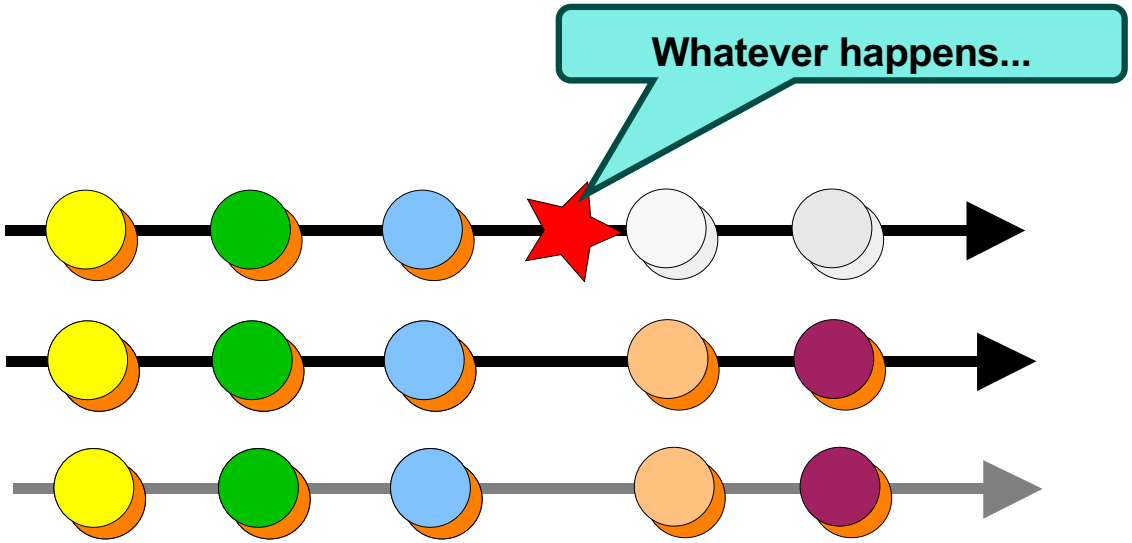
- **Error detection**
  - *Detecting the error after it occurs to confine it to avoid propagation and trigger error recovery mechanisms*
  - **E.g.: modified files or messages; phony OS account; sniffer in operation; crashing on logic bomb**
- **Error recovery**
  - *Recovering from the error to provide correct service despite the error*
  - **E.g.: recovering from effects of intrusions**



©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Processing the errors deriving from intrusions

- **Error Masking**
  - *Redundancy allows providing correct service without noticeable glitch*
  - **Examples:**
    - » **voting, Byzantine fault-tolerant replication, secret sharing**
    - » **sensor correlation (agreement on imprecise values)**



©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Outline

- **Preliminary Observations**
- **Concepts**
  - **Fault models**
  - **Methodologies**
  - **Error Processing**
- **Framework**
- **Strategies**

A high-level view of the tools available to build intrusion-tolerant systems

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Intrusion tolerance frameworks

Secure and fault-tolerant communication

- **Concerns the body of protocols ensuring intrusion-tolerant communication:**
  - **Secure channels (e.g., TLS/SSL) and secure envelopes (e.g., PGP)**
  - **Fault-tolerant group communication (e.g., total order multicast)**
- **What should be the classes of failures of communication network components?**
  - **Baseline: arbitrary faults (maximum coverage)**
  - **Some components can be trusted-trustworthy or fail only in an omissive way (fail-controlled), but the coverage of these assumptions should be ensured**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Intrusion tolerance frameworks

Software-based intrusion tolerance

- **Software-based fault tolerance**
  - Tolerating hardware faults using software techniques (e.g., replication)
- **Software fault tolerance**
  - Tolerating software design faults (i.e., bugs) by design diversity
- **Main challenge: common mode faults**
  - How much diversity to be effective? What about the costs?
  - Common mode vulnerabilities (like bugs) can be avoided using diverse components
  - Attacks can also be avoided since the attacker should have to master different platforms to compromise the system

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Intrusion tolerance frameworks

Hardware-based intrusion tolerance

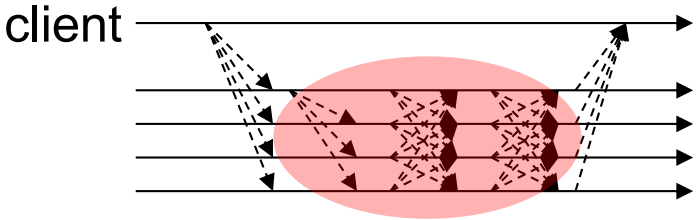
- **Means to construct fail-controlled components**
  - Contributes to establishing improved levels of trustworthiness
  - Certain components can be built in such a way that it is very unlikely that someone compromises them (e.g., Intel SGX)
  - If they fail, it will be only by crashing (i.e., the attacker turns off the machine or shuts down the module)
- **Distributed algorithms that tolerate arbitrary faults are expensive in both resources and time**
  - With fail-controlled components, we can make them more affordable

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

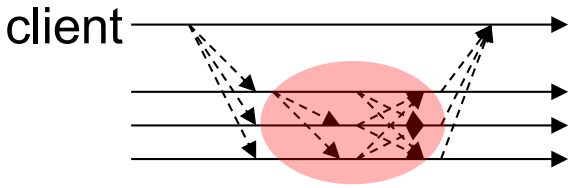
Intrusion tolerance frameworks

Hardware-based intrusion tolerance (an example)

- **Byzantine agreement on non-synchronous systems**
  - **Arbitrary failures: requires *3f+1* processes and at least three communication steps in the normal case**



  - **Arbitrary failures plus each process equiped with a TPM: it is possible to solve the problem with *2f+1* processes and two communication steps**



©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Intrusion tolerance frameworks

Auditing

- Routinely logging actions and events is a good management procedure
- Allows "a posteriori" diagnosis of problems and their causes

- **Audit trails are a crucial framework in security**
  - trace back the events and actions associated with a given time interval, subject, object, service, or resource
- **Logs should be tamperproof**
  - which they are not in many systems
- **Part of the appealing of blockchains is their use as ledgers for registering transactions**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Intrusion tolerance frameworks

Intrusion Detection (ID)

- Encompasses all kinds of attempts to detect the presence or the likelihood of an intrusion
- Can be performed in real-time, or off-line

- **ID systems are supervision systems that follow and log system activity to detect and react (preferably in real-time) against any or all off:**
  - attacks (e.g., port scan detection),
  - vulnerabilities (e.g., scanning),
  - and intrusions (e.g., correlation engines).

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Outline

- **Preliminary Observations**
- **Concepts**
  - **Fault models**
  - **Methodologies**
  - **Error Processing**
- **Frameworks**
- **Strategies**

What are the goals that should drive the design of an intrusion-tolerant system?

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.

Intrusion tolerance strategies

System construction

- **Fault avoidance + Fault tolerance**

System goals (pick one or more of these)

- **Confidential operation**
- **Perfect non-stop operation**
- **Reconfigurable operation**
- **Recoverable operation (e.g., disaster recovery)**
- **Fail-safe**

©2010-25 P. Veríssimo and A. Bessani – All rights reserved.